南方科技大学

LAB 3: Preprocessing & Output Layers

COE201, Spring 2026

2026-03-09

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

Input Preprocessing 1

Word Representations 2

Tokenization and Embedding Matrix 5

Output Layers of Language Models . . . 8

From Hidden States to Probabilities . 9

Decoding Strategies 12

Full LLM Pipeline 21

Summary: The Full LLM Pipeline . 22

Lab Tasks 23

Pre-trained Embeddings: GloVe . . . 24

Lab Exercises 25

Input Preprocessing

Mapping raw text to dense vectors.

Word Representations

One-Hot Vectors

The simplest way to represent words is using **One-Hot Vectors**. If our vocabulary has N words, each word is an N-dimensional vector with a single 1 and the rest 0.

```python
# Vocabulary: ["cat", "dog", "car"]
cat = [1, 0, 0]
dog = [0, 1, 0]
car = [0, 0, 1]
```

Problems with One-Hot Vectors:

- **High Dimensionality & Sparsity**: A vocabulary of 50,000 words means vectors of length 50,000, mostly filled with 0s.
- **No Semantic Meaning**: The distance between "cat" and "dog" is the same as "cat" and "car". They are all orthogonal (dot product $= 0$).

Word Representations

Dense Embeddings (Word2Vec)

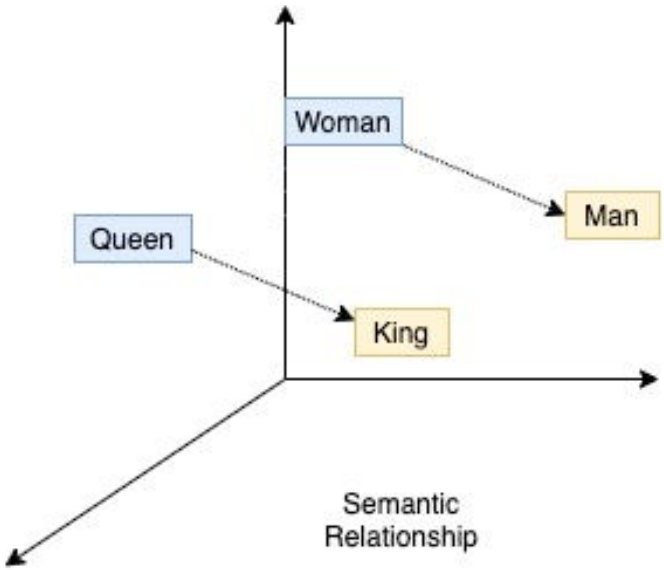
Instead of sparse vectors, we use **dense vectors** of fixed size (e.g., 256 or 512 dimensions) where similar words have similar vectors.

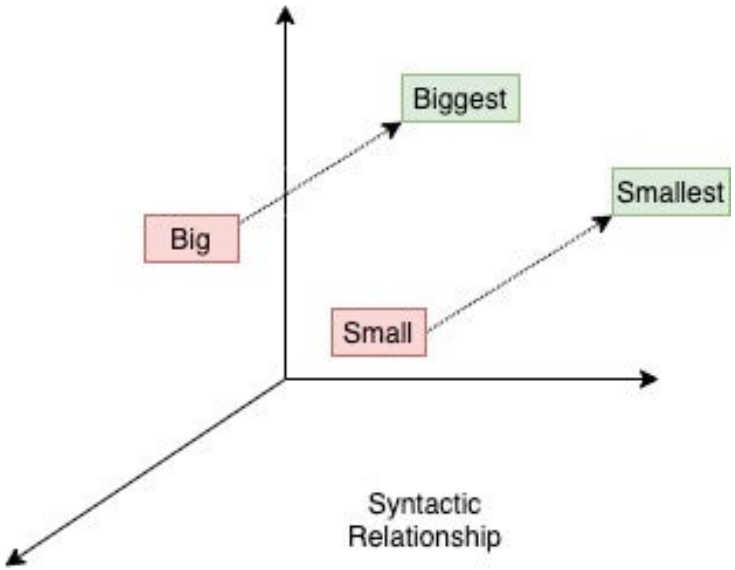
- This allows the model to capture **semantic meaning**. (e.g., $\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$).
- Dense embeddings are usually learned from data (e.g., Word2Vec, GloVe, or jointly trained with the language model).

One-Hot ("cat")	**Dense Embedding ("cat")**
$[0, 0, 0, \ldots, 1, 0, 0]$	$[0.82, -0.45, 0.12, 0.99, \ldots, -0.21]$

Dense vectors have non-zero cosine similarity for semantically related words!

Visualization of Dense Word Embeddings





Vector Analogies: Dense embeddings capture relationships through vector offsets:

Semantic (Gender): **King** − **Queen** ≈ **Man** − **Woman**

Syntactic (Comparative): **Biggest** − **Big** ≈ **Smallest** − **Small**

Tokenization and Embedding Matrix

Tokenization

Before we can embed words, we need to split sentences into units called **tokens**.

- **Word-level**: "the quick brown fox" -> `["the", "quick", "brown", "fox"]`
- **Character-level**: "hello" -> `["h", "e", "l", "l", "o"]`
- **Subword-level (BPE)**: "unhappiness" -> `["un", "happi", "ness"]`

After splitting, we map each token to an Integer ID using a predefined Vocabulary table.
`"the" -> 1, "quick" -> 2, "brown" -> 3`

Why LLMs use BPE?

- **Balances**: Keeps vocabulary size manageable. (Compared to word-level)
- **Robust**: Rare words are broken into known sub-units. (e.g. "biochemistry" -> "bio", "chem", "istry")

(actual BPE splits depend on the learned vocabulary list)

Tokenization and Embedding Matrix

PyTorch nn.Embedding

In PyTorch, we convert Token IDs to dense vectors using an `nn.Embedding` layer. It acts as a **lookup table**.

```python
import torch
import torch.nn as nn

# Vocab size = 5000, Embedding Dim = 128
embedding_layer = nn.Embedding(num_embeddings=5000, embedding_dim=128, padding_idx=0)

token_ids = torch.tensor([[1, 2, 3]]) # Batch of 1 text, 3 tokens
dense_vectors = embedding_layer(token_ids)

print(dense_vectors.shape) # (1, 3, 128)
```

Tokenization and Embedding Matrix

Special Tokens

Modern tokenizers include **special tokens** to handle structural information and edge cases.

- **<PAD> (Padding)**: Makes all sequences in a batch the same length. Usually ignored by the loss function and attention mechanism.
- **<BOS> / <s> (Beginning of Sentence)**: Signals the start of text. Essential for autoregressive generation to "kick off" the model.
- **<EOS> / </s> (End of Sentence)**: Tells the model when to stop generating. Without this, the model might loop forever!
- **<UNK> (Unknown)**: Replaces words that were not seen during training (not in the vocabulary).

Output Layers of Language Models

Converting hidden states to word probabilities.

From Hidden States to Probabilities

Logits

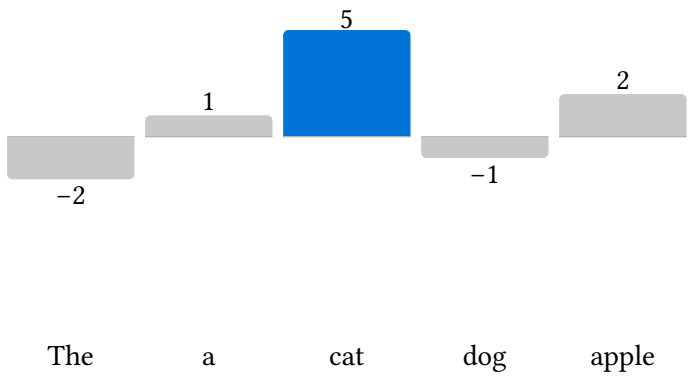
Language model output is projected to vocabulary size via a Linear layer.

- These raw, unnormalized scores are called **Logits**.
- Logits can be any real number ([− inf, inf]).

Code example:

```python
class LanguageModel(nn.Module):
  def __init__(self, ...):
    ... # Other layers
    self.output_layer = nn.Linear(128, 5000)  # 5000: vocab size
  def forward(self, hidden_state):
    ... # Other layers
    # hidden_state shape: (batch_size, seq_len, hidden_dim)
    logits = self.output_layer(hidden_state)
    return logits
```

Raw Logits (z)



From Hidden States to Probabilities

Softmax

To convert logits into a valid probability distribution (all positive, summing to 1), we apply the **Softmax** function:

$$\mathrm{Softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

```python
import torch.nn.functional as F

logits = torch.tensor([[-2.0, 1.0, 5.0, -1.0, 2.0]])
probs = F.softmax(logits, dim=-1)
# probs: [[0.0009, 0.0177, 0.9659, 0.0024, 0.0131]]
```

From Hidden States to Probabilities

Visualizing Softmax

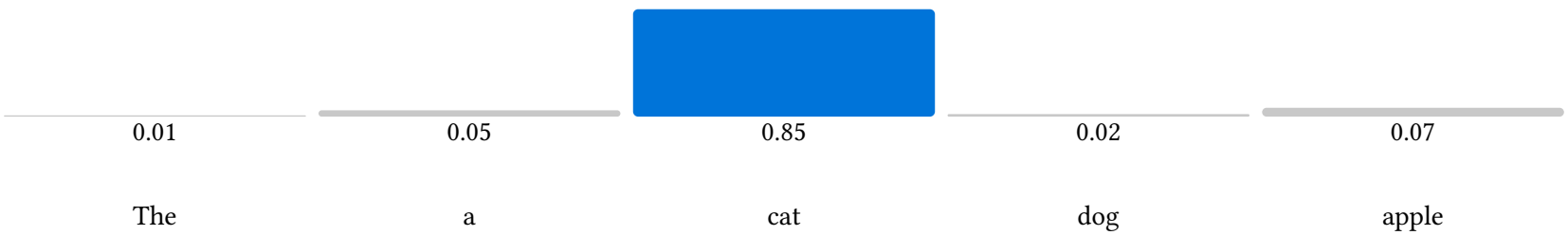
Logits (z) $[-2.0, 1.0, \mathbf{5.0}, -1.0, 2.0]$

$\downarrow$

Softmax

$\downarrow$

Probability Distribution (P)



Decoding Strategies

Decoding and Sampling

How do we pick the next word using these probabilities?

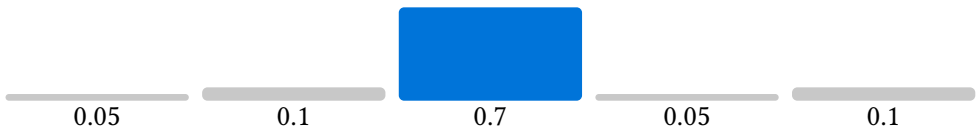
1. **Greedy Decoding**: Always pick the highest probability word (argmax).
   - Can lead to repetitive, uncreative text.
2. **Random Sampling**: Sample from the probability distribution (e.g., `torch.multinomial`).
   - Introduces randomness, making text more natural.

Decoding Strategies

Visualizing Decoding Strategies

Greedy Decoding

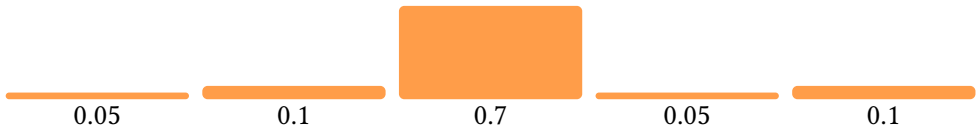
Pick the **highest** probability



Result: Always "cat"

Random Sampling

Pick **any** based on probability



Result: "cat" (70%), "dog" (10%)...

"Greedy is predictable; Sampling is creative."

Decoding Strategies

Why Greedy Fails: Local vs. Global

The probability of a sequence is the product of **conditional probabilities**:

$$P(w_1, w_2, w_3) = P(w_1) \times P(w_2 \mid w_1) \times P(w_3 \mid w_1, w_2)$$

Example: Generating $L = 3$ words ($w_{\{<t\}}$ is the previous context):

Greedy Path (w_t is local max)

- Step 1: $P(\text{The} \mid \text{Start}) = \mathbf{0.6}$
- Step 2: $P(\text{quick} \mid \text{The}) = \mathbf{0.4}$
- Step 3: $P(\text{fox} \mid \text{The quick}) = \mathbf{0.4}$

Result: **"The quick fox"**

$P = 0.6 \times 0.4 \times 0.4 = \mathbf{0.096}$

Global Optimum (higher total P)

- Step 1: $P(\text{A} \mid \text{Start}) = 0.4$
- Step 2: $P(\text{very} \mid \text{A}) = 0.8$
- Step 3: $P(\text{good} \mid \text{A very}) = 0.9$

Result: **"A very good"**

$P = 0.4 \times 0.8 \times 0.9 = \mathbf{0.288}$

Greedy's Mistake: Maximizes the **current** prob but ignores those choices constrain the future probabilities.

Decoding Strategies

Advanced Decoding: Beam Search

Greedy decoding (beam width $= 1$) only considers the **local** best choice, which might lead to a poor **global** result.

Beam Search (width $= k$):

- Keeps the top k most likely sequences at each step.
- At step $t + 1$, expands all k paths and keeps the top k **cumulative** probabilities.
- **Benefit**: Explores a larger search space than Greedy.

Decoding Strategies

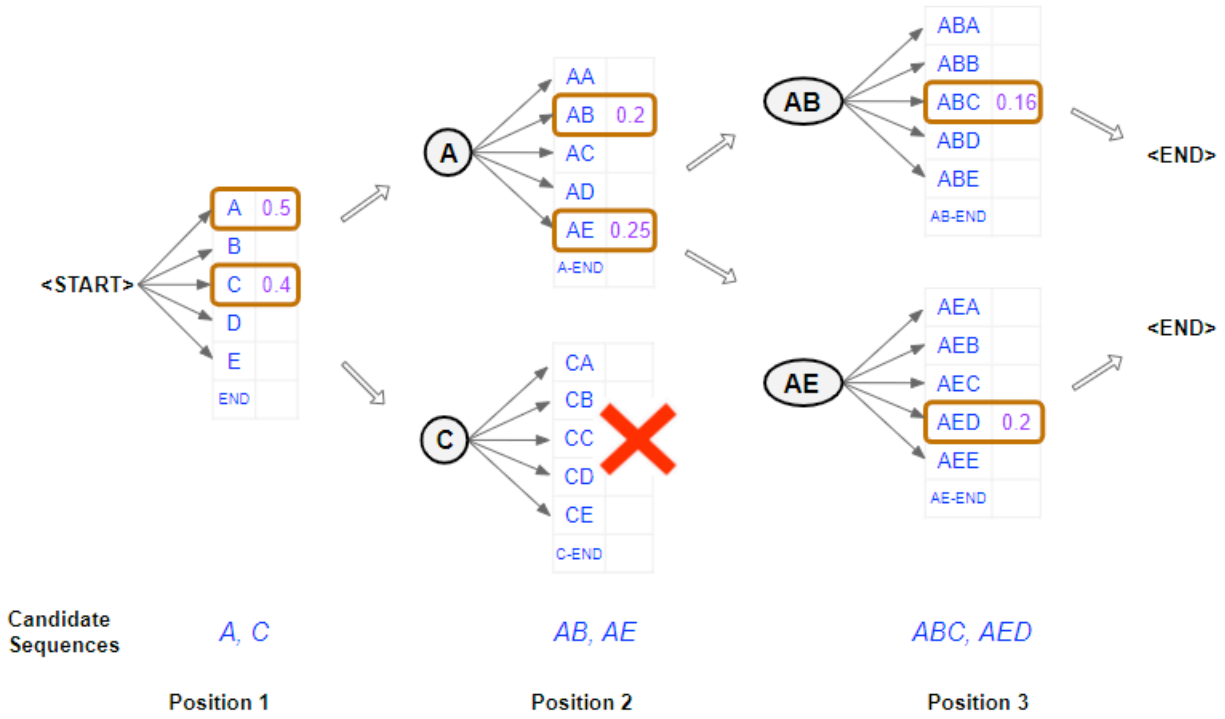
Beam Search: Computational Process

1. **Expansion**: For each of the k active beams, the model predicts the next token distribution over the **entire** vocabulary V.
2. **Scoring**: Compute the **cumulative log-probability** for all $k \times V$ possible candidates.
3. **Selection**: Sort all candidates and keep only the **top k** sequences for the next step.

$$\boxed{k \text{ Beams}} \xrightarrow{\text{Expand}} \boxed{k \times V \text{ Candidates}} \xrightarrow{\text{Selection}} \boxed{\text{Next } k \text{ Beams}}$$

Resulting Complexity: $O(L \times k \times V)$. *k times more expensive than Greedy!*

Beam Search: The Search Tree



Only the top k paths are kept at each depth.

Decoding Strategies

Why not Beam Search?

While Beam Search finds higher probability sequences, it is often **avoided** for Chatbots (e.g., ChatGPT).

- **High Computational Complexity**: $O(L \times k \times V)$ is much slower than Greedy/Sampling decoding (which is $O(L \times V)$).
- **Generic Output**: High-probability text is often safe but repetitive (e.g., "I don't know," "I think that...").
- **Lack of Diversity**: The k beams often become almost identical.

Solution: Modern LLMs prefer **Sampling with Temperature** (Top-p, Top-k) to maintain creativity and variety.

Decoding Strategies

Temperature Scaling

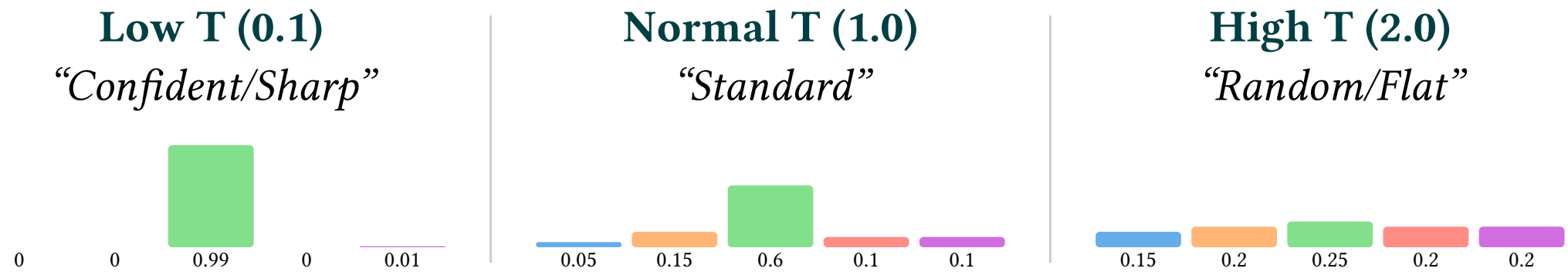
We can adjust the "randomness" by dividing logits by a **Temperature (T)** before softmax.

$$\text{Softmax}\left(\frac{z_i}{T}\right)$$

- $\boldsymbol{T = 1.0}$: Standard Softmax.
- $\boldsymbol{T < 1.0}$ (e.g., 0.5): Makes the distribution "sharper". High probabilities get higher, making text more deterministic and conservative.
- $\boldsymbol{T > 1.0}$ (e.g., 2.0): Makes the distribution "flatter". Probabilities become more uniform, making text more random and creative.

Decoding Strategies

Visualizing Temperature Scaling



$T \downarrow$ Distribution becomes **Sharper** (Focus on top choice)

$T \uparrow$ Distribution becomes **Flatter** (Uniform randomness)

Full LLM Pipeline

Summary: The Full LLM Pipeline

Input Text → **Preprocessing** → **Embedding** → Continue ⟹

"The cat..." | Tokens & IDs | Dense Vectors

⟹ **LLM (Model)** → **Output Layer** → **Decoding**

Hidden States | Logits & Probs | Next Token ID

Autoregressive Loop

Autoregressive Loop:

The predicted token is added back to the input and the process repeats!

(until `<|EOS|>` special token is generated or max length is reached)

Lab Tasks

Hands-on with embeddings and output layers.

Complete the lab exercises.

Pre-trained Embeddings: GloVe

GloVe (Global Vectors for Word Representation) is a popular pre-trained embedding model developed by Stanford.

GloVe: Global Vectors for Word Representation

Jeffrey Pennington, Richard Socher, Christopher D. Manning
Computer Science Department, Stanford University, Stanford, CA 94305
jpennin@stanford.edu, richard@socher.org, manning@stanford.edu

- **GloVe**: Learns by factorizing a global **word-word co-occurrence matrix**.
  - It captures the ratio of co-occurrence probabilities between words.
  - "Ice" is more likely to co-occur with "solid" than "steam".
  - "Water" co-occurs with both "solid" and "steam".
- Vocabulary size: **400,000** unique words.
- Embedding dimension: **50**.

Lab Exercises

1. **Task 1: Real Word Embeddings** (`task1_embeddings.py`)
   - Implement **Cosine Similarity** manually.
   - Verify semantic clustering and **Word Analogies** (e.g., $\mathbf{Queen} - \mathbf{Woman} + \mathbf{Man} \approx \mathbf{King}$).
2. **Task 2: The Embedding Matrix** (`task2_tokenization.py`)
   - Explore a **Real-world Vocabulary** (GloVe) with 400,000+ words.
   - Implement **Manual Lookup** and proof its equivalence to **One-hot multiplication**.
3. **Task 3: Logits, Softmax & Top-K** (`task3_softmax.py`)
   - Implement **Softmax** manually with numerical stability trick ($x - \max(x)$).
   - Observe **Temperature** scaling effects ($T = 0.1$ vs $T = 5.0$).
   - Implement **Top-K Filtering** and sample from the filtered distribution.

Lab Exercises

1. **Task 4 (Optional): Beam Search Implementation** (`task4_beam_search.py`)
   - Implement a basic two-step Beam Search walkthrough to understand the expansion/selection cycle.
   - Discussion: Derive the $O(L \times k \times V)$ complexity and discuss modern LLM optimization (from knowledge learned before).

Thanks for Listening!